

# Generative Recommendation for Large-Scale Advertising

Ben Xue\*, Dan Liu\*, Lixiang Wang\*, Mingjie Sun\*, Peng Wang\*, Pengfei Zhang\*, Shaoyun Shi\*, Tianyu Xu\*, Yunhao Sha\*, Zhiqiang Liu\*, Bo Kong, Bo Wang, Hang Yang, Jieting Xue, Junhao Wang, Shengyu Wang, Shuping Hui, Wencai Ye, Xiao Lin, Yongzhi Li, Yuhang Chen, Zhihui Yin, Quan Chen, Shiyang Wen, Wenjin Wu, Han Li, Guorui Zhou, Changcheng Li, Peng Jiang†, Kun Gai  
Kuaishou Technology  
Beijing, China

## Abstract

Generative recommendation has recently attracted widespread attention in industry due to its potential for scaling and stronger model capacity. However, deploying real-time generative recommendation in large-scale advertising requires designs beyond large-language-model (LLM)-style training and serving recipes. We present a production-oriented generative recommender co-designed across architecture, learning, and serving, named **GR4AD** (Generative Recommendation for **AD**vertising). As for tokenization, GR4AD proposes UA-SID (Unified Advertisement Semantic ID) to capture complicated business information. Furthermore, GR4AD introduces LazyAR, a lazy autoregressive decoder that relaxes layer-wise dependencies for short, multi-candidate generation, preserving effectiveness while reducing inference cost, which facilitates scaling under fixed serving budgets. To align optimization with business value, GR4AD employs VSL (Value-Aware Supervised Learning) and proposes RSPO (Ranking-Guided Softmax Preference Optimization), a *ranking-aware, list-wise* reinforcement learning algorithm that optimizes value-based rewards under list-level metrics for continual online updates. For online inference, we further propose dynamic beam serving, which adapts beam width across generation levels and online load to control compute. Large-scale online A/B tests show up to 4.2% ad revenue improvement over an existing DLRM-based stack, with consistent gains from both model scaling and inference-time scaling. GR4AD has been fully deployed in Kuaishou advertising system with over 400 million users and achieves high-throughput real-time serving.

## Keywords

Generative Recommendation, Advertising

## 1 Introduction

Recent industrial efforts on recommender systems have started converting deep learning recommendation models (DLRMs) into generative recommenders, driven by their potential for scaling and stronger model capacity, with significant gains across a range of business scenarios [12, 34–36, 39, 40].

Despite encouraging progress, deploying real-time generative recommendation models in large-scale advertising systems remains challenging. The unique characteristics of advertising recommendation make a direct reuse of LLM techniques insufficient in several aspects. (1) **Advertisement Tokenization**. Tokenization is fundamental to LLM generation and particularly challenging for

advertising, where short-video creatives fuse video attributes, product details, and B2B advertiser metadata. Prior work incorporates multimodal cues or pretrained MLLMs for content understanding, but no end-to-end, fine-tuned advertisement LLM embedding exists. Furthermore, platforms expose salient business signals (e.g., conversion type, ads account) that are not readily captured by semantic content, so jointly modeling multimodal, multi-granularity features for user interest and business value remains a core challenge. (2) **Learning Paradigm**. Advertising recommendation optimizes *ranked lists* under business objectives (e.g., eCPM) and list-wise metrics (e.g., NDCG), which are not well captured by per-item supervision. Existing approaches, largely following LLM-style training recipes, lack a ranking-aware, list-wise learning design tailored to online advertising learning [3, 28, 38, 41]. (3) **Real-Time Serving**. Generative recommendation does not remove the stringent serving constraints of DLRM-based stacks: the system must produce multiple high-quality candidates under high traffic and strict latency budgets. This setting differs fundamentally from interactive LLM usage, where decoding a single response can tolerate substantially longer latency. Beyond optimizations borrowed from LLM inference, serving-time efficiency for real-time multi-candidate generation in advertising remains under-explored in a systematic way.

To address the above gaps, we present a production-oriented generative recommender designed for real-time, large-scale advertising, named **GR4AD** (Generative Recommendation for **AD**vertising). GR4AD adopts a recommendation-native co-design across representation, learning, and serving. (1) **Unified Advertisement Semantic ID**. We propose UA-SID, derived from a fine-tuned MLLM embedding trained on real-world advertisement creatives via instruction tuning and co-occurrence learning to capture complicated information. Furthermore, we introduce MGMR (Multi-Granularity-Multi-Resolution) RQ-Kmeans quantization method to model non semantic information, substantially reduce SID collisions, and improve codebook utilization. (2) **Value-Aware Online Learning**. To align optimization with business value in non-stationary markets, we design VSL (Value-Aware Supervised Learning) to learn the user-interest distribution, and propose RSPO (Ranking-Guided Softmax Preference Optimization), a ranking-guided, list-wise RL algorithm that explicitly optimizes list-level objectives. We further develop an online learning framework that tightly integrates VSL and RL for continual, frequent model updates. (3) **Recommendation-Oriented Efficiency Optimizations**. To satisfy strict real-time constraints, we systematically optimize decoding and serving. We propose a new LazyAR decoder architecture to relax layer-wise autoregressive dependencies and boost decoding throughput without hurting effectiveness. Beyond LLM-style optimizations, we further

\* Authors contributed equally and are listed in alphabetical order.

† Corresponding author.

introduce recommendation-specific serving techniques, including Dynamic Beam Serving (DBS)—with Dynamic Beam Width (DBW) and Traffic-Aware Adaptive Beam Search (TABS)—as well as a short time-to-live (TTL) cache.

Online A/B tests show that GR4AD delivers up to 4.2% ad revenue improvement compared to the existing DLRM-based stack, with consistent gains from both model scaling and inference-time scaling. With systematic efficiency optimizations in architecture and serving, GR4AD achieves <100ms latency and 500+ QPS per L20 under practical resource budgets, and has been fully deployed in Kuaishou advertising system serving over 400 million users.

## 2 Related Works

### 2.1 Generative Recommendation

Recent advances in generative models [1, 10, 11, 14, 17], particularly Large Language Models (LLMs) [1], have catalyzed a new paradigm in recommendation systems based on end-to-end generation. A representative line of work is Semantic ID-based generative recommendation [16], which encodes items as discrete semantic identifiers (SIDs) and formulates recommendation as next-token prediction. TIGER [25] is a seminal work that introduces hierarchical SIDs via residual quantization of item features and models recommendation as a generative retrieval task using an encoder-decoder Transformer. Building on this foundation, subsequent studies such as LC-Rec [38], LETTER [28], and OpenOneRec [41] improve scalability and generalization by aligning item identifiers with collaborative semantics through two-stage training. OneRec [39, 40] unifies retrieval and ranking within a single generative model and has shown strong performance in large-scale industrial recommendation, while GPR [36] and OneSearch [3] extend the generative paradigm to advertising and e-commerce search, respectively. To address the inference inefficiency of autoregressive generation, RPG [15] and NEZHA [31] propose parallel and hyperspeed decoding mechanisms, and MMQ [32] further generalizes generative recommendation to multimodal settings via discrete quantization.

Despite these advances, existing generative recommenders largely lack architectures and learning strategies specifically designed for advertising systems, particularly under online learning constraints.

### 2.2 Reinforcement Learning

Reinforcement learning has become a central paradigm for LLM preference optimization. Early RLHF [22] formulates alignment as policy optimization over a learned reward model, but its instability and engineering cost have motivated simpler preference-based alternatives. DPO [24] removes explicit reward modeling by optimizing directly over preference pairs, followed by variants such as SimPO [21], GRPO [26], SAPO [9], and GDPO [18], which further improve stability or multi-objective learning. However, these methods are largely offline, relying on static preference data or fixed rollouts, and mainly address distributional alignment rather than continual learning in non-stationary environments.

In recommender, search, and advertising systems, reinforcement learning has been increasingly adopted to align large generative models with user behavior and business objectives. Works such as OneRec [39, 40], OneSearch [3], and GPR [36] adapt policy optimization via reward shaping, preference weighting, or hierarchical

objectives, but often depend on multi-stage pipelines or naive combinations of SFT and RL, limiting adaptability to streaming settings. To address this, recent studies including SRFT [8], CHORD [37], and the unified policy optimization framework in [20] propose principled joint SFT-RSPO optimization through dynamic weighting or shared objectives. Building on these ideas, we design an efficient SFT-RSPO integration framework tailored to streaming recommender scenarios.

### 2.3 Semantic IDs

In traditional advertising systems, each item is assigned a sequentially increasing ID, which suffers from data sparsity and hampers cold-start. In contrast, Semantic IDs leverage LLMs to encode item content (text, video, etc.) into embeddings, which are then hierarchically quantized into structured identifiers. Quantization methods fall into two main categories: RQ-VAE-based and clustering-based. TIGER [25] uses RQ-VAE to recursively quantize residuals for hierarchical IDs. FORCE [7] introduces a cross-entropy equalization loss for balanced codebook usage. PLUM [13] uses multi-resolution codebooks and progressive masking for clear hierarchical structure. COST [42] proposes a contrastive learning framework to optimize the codebook for more discriminative IDs. QARM [19] adopts RQ-Kmeans with cardinality constraints to dynamically regulate codebook usage, improving balance and efficiency. Furthermore, methods like ColaRec [30], SEATER [27], and TokenRec [23] integrate collaborative filtering: they first derive collaborative signal-enriched embeddings from user-item interactions, then encode them into discrete semantic IDs, injecting collaborative inductive bias into the generative recommendation framework.

## 3 Methodology

In this section, we detail the model architecture and training paradigm of GR4AD. Figure 1 provides an overview of the method.

### 3.1 Unified Advertisement Semantic ID

In generative recommendation, Semantic IDs play a role analogous to tokens in large language models, where both representation granularity and quantization quality critically affect downstream performance. We therefore design *Unified Advertisement Semantic IDs (UA-SID)* which is derived from an end-to-end finetuned advertisement LLM embedding model, as illustrated in the Figure 2.

#### 3.1.1 Unified Advertisement Embedding (UAE).

*Instruction Tuning (IT).* To capture heterogeneous content formats in advertising (video, product and advertiser), we employ instruction-based fine-tuning of an LLM to instill ad-understanding capabilities across diverse aspects and scenarios. For example, for a live-stream host we prompt the model to analyze their profile and geographic information, while for ordinary off-platform advertisers we direct the model to focus on the product's industry and brand information. We designed 6 instructions in total to comprehensively cover Kuaishou's ad types (presented in the appendix).

*Co-occurrence Learning (CL).* To incorporate collaborative signals from user behavior, we augment representation learning with a co-occurrence contrastive objective. Item co-occurrence strengths are estimated using the Swing method [33]. We further augment

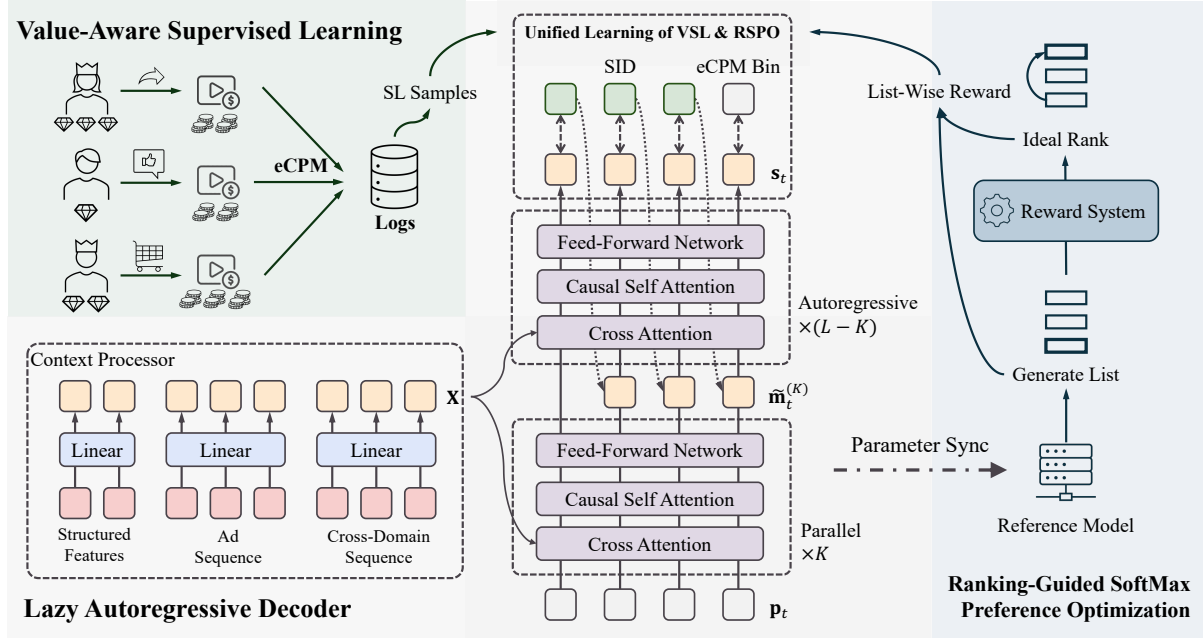


Figure 1: Overview of our proposed GR4AD: model architecture and learning algorithm.

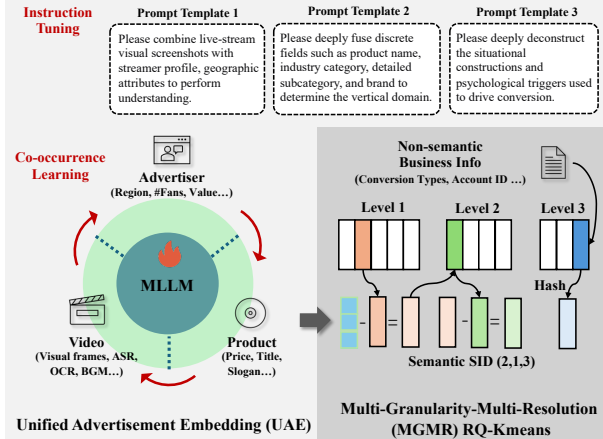


Figure 2: Illustration of Unified Advertisement Semantic ID.

positive samples  $\mathcal{P}_i$  with respect to each item triplets (Video, Product, Advertiser), where the co-occurring pair is treated as positives, while other in-batch samples serve as negatives. We adopt the InfoNCE:

$$\mathcal{L}_{\text{NCE}}(i) = -\log \frac{\sum_{j \in \mathcal{P}_i} \exp(\mathbf{z}_i^\top \mathbf{z}_j / \tau)}{\sum_{k \neq i} \exp(\mathbf{z}_i^\top \mathbf{z}_k / \tau)}, \quad (1)$$

where  $\mathbf{z}$  represents the last hidden states of MLLM.

**3.1.2 Multi-Granularity-Multi-Resolution (MGMR) RQ-Kmeans.** The stability and balance of the UA-SID codebook are critical for effective generative retrieval. Lower codebook utilization and higher SID collision ratio can degrade downstream performance.

Standard RQ-Kmeans with fixed codebook size often suffers from low codebook utilization [19]. Inspired by [13], we adopt a *Multi-Resolution (MR)* RQ-Kmeans scheme. To better preserve semantic separability, lower levels use larger codebooks to capture dominant

factors early, while higher levels model lower-entropy residuals. Balanced K-means clustering is applied at each level to improve codebook utilization.

SID collisions are also acute in advertising, identical-content ads can exhibit entirely different delivery trajectories if advertisers target distinct conversion types. In fact, ad systems routinely expose such *Multi-Granularity (MG)*, predominantly numeric signals that lack conventional semantics. Therefore, we replace vector quantization at the final layer with a hash-based numeric mapping derived from non-semantic features (e.g., item/account IDs, conversion types), which markedly improves global balance and reduces collisions with negligible extra quantization error.

Finally, each item is mapped to a discrete UA-SID sequence

$$\mathbf{y} = (s_1, s_2, \dots, s_T), \quad s_t \in \mathcal{V}_t, \quad (2)$$

where  $s_t$  denotes the token at level  $t$ ,  $T$  is the UA-SID depth (typically small), and  $\mathcal{V}_t$  is the vocabulary at level  $t$ .

### 3.2 Lazy Autoregressive Decoder

For structured features and user interaction sequences, we follow LazyDecoder [40] and adopt a lightweight linear context processor to efficiently model these heterogeneous contexts, whose output is denoted as  $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_S)$  with  $\mathbf{x}_i \in \mathbb{R}^d$ , where  $S$  is the context length and  $d$  is the hidden size. The decoder then generates the target UA-SID  $\mathbf{y}$ .

*Vanilla Autoregressive Decoding.* Prior work commonly uses a standard autoregressive decoder that factorizes

$$p(\mathbf{y} | \mathbf{X}) = \prod_{t=1}^T p(s_t | s_{<t}, \mathbf{X}), \quad (3)$$

and feeds the embedding of the previous-level UA-SID back to the *first* decoder layer at the next decoding step, as shown in Figure 3(a). Concretely, let  $\mathbf{s}_t \in \mathbb{R}^d$  be the embedding of  $s_t$  (and  $\mathbf{s}_0 = \text{BOS}$ ) and

$\mathbf{p}_t$  be the position embedding of step  $t$ . A naive decoder initializes the step- $t$  state with  $\mathbf{h}_t^{(0)} = \mathbf{s}_{t-1} + \mathbf{p}_t$  and applies  $L$  decoder layers:

$$\mathbf{h}_t^{(\ell)} = \text{Dec}^{(\ell)}(\mathbf{h}_t^{(\ell-1)}, \mathbf{X}), \quad \ell = 1, \dots, L, \quad (4)$$

followed by a classifier  $p(s_t | \cdot) = \text{Softmax}(\mathbf{W}_t \mathbf{h}_t^{(L)})$ .

**LazyAR: Late-Inject Autoregression.** In the experiments, we observed that the first-level UA-SID ( $s_1$ ) typically has the largest loss and is the most important to learn, yet contributes little to beam-search cost: decoding starts from BOS, so the effective beam for  $s_1$  is 1, while beams become much larger for later levels ( $t \gg 1$ ). As a result, most decoding compute is spent on later levels, which are empirically easier. Motivated by this mismatch between learning difficulty and inference cost, we ask whether we can reduce the decoding compute for later-level UA-SID without affecting the inference of the first level.

A straightforward approach is to add extra shallow decoders for later UA-SID levels (e.g., DeepSeek MTP [5], shown in Figure 3(b)), but this (i) introduces additional parameters, (ii) prevents early decoder layers from directly participating in later-level inference and (iii) empirically decreases the recommendation performance. Instead, we propose **LazyAR** (Lazy AutoRegression), which delays the dependence on  $s_{t-1}$  to an intermediate layer. Detailedly, given a chosen  $K$  ( $1 \leq K < L$ ), for each UA-SID level  $t$ , LazyAR computes the first  $K$  layers *without* conditioning on  $s_{t-1}$ :

$$\mathbf{m}_t^{(0)} = \mathbf{p}_t, \quad (5)$$

$$\mathbf{m}_t^{(\ell)} = \text{Dec}^{(\ell)}(\mathbf{m}_t^{(\ell-1)}, \mathbf{X}), \quad \ell = 1, \dots, K, \quad (6)$$

then injects the previous-level UA-SID embedding at layer  $K$  via a fusion operator:

$$\tilde{\mathbf{m}}_t^{(K)} = \text{Fuse}(\mathbf{m}_t^{(K)}, \mathbf{s}_{t-1}), \quad (7)$$

and applies the remaining  $L - K$  layers autoregressively:

$$\mathbf{h}_t^{(\ell)} = \text{Dec}^{(\ell)}(\mathbf{h}_t^{(\ell-1)}, \mathbf{X}), \quad \ell = K + 1, \dots, L, \quad (8)$$

with  $\mathbf{h}_t^{(K)} \triangleq \tilde{\mathbf{m}}_t^{(K)}$  and

$$p(s_t | s_{<t}, \mathbf{X}) = \text{Softmax}(\mathbf{W}_t \mathbf{h}_t^{(L)}). \quad (9)$$

We implement  $\text{Fuse}(\cdot)$  as a lightweight gated projection:

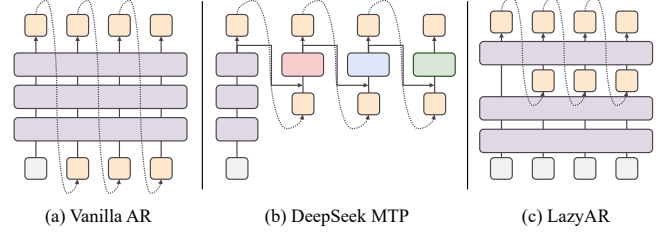
$$\text{Fuse}(\mathbf{m}, \mathbf{s}) = \mathbf{W}_f[\mathbf{m} \odot (\mathbf{W}_g \mathbf{s}); \mathbf{s}], \quad (10)$$

where  $[\cdot; \cdot]$  is concatenation and  $\odot$  is element-wise product. Each decoder layer  $\text{Dec}^{(\ell)}(\cdot)$  follows OneRecV2 [40], consisting of a cross-attention between  $\mathbf{m}_t^{(\ell)}/\mathbf{h}_t^{(\ell)}$  and  $\mathbf{X}$ , a self-attention module over decoder states, and a feed-forward network. All submodules are equipped with pre-layer normalization to stabilize training and facilitate deeper architectures.

**Why LazyAR is faster.**  $\mathbf{m}_t^{(K)}$  does not depend on  $s_{t-1}$ , so the first  $K$  layers can be computed in parallel for all levels and reused across beams:

$$\{\mathbf{m}_t^{(K)}\}_{t=1}^T = \text{Dec}^{(1:K)}(\{\mathbf{p}_t\}_{t=1}^T, \mathbf{X}). \quad (11)$$

Only the remaining  $L - K$  layers (Eq. (8)) require autoregressive dependency, reducing sequential work per level during beam search. This is important because, at later UA-SID levels, decoding must



**Figure 3: Comparison of Vanilla AR, DeepSeek MTP, LazyAR.**

be performed for a large number of beam candidates in parallel, making the cost of the autoregressive component a major bottleneck in overall decoding efficiency.

**Why LazyAR preserves performance.** (1) The decoding process for the first-level UA-SID remains unchanged:  $s_1$  is still generated from a initialized embedding through the full  $L$  decoder layers, ensuring no degradation on the hardest level. (2) For later steps, LazyAR directly leverages the representations produced by the first  $K$  layers. These intermediate states are not arbitrary – they can reason in latent space and encode useful signals about plausible candidates at level  $t$ . To encourage the first  $K$  layers to learn richer and more predictive latent representations, we introduce an MTP-style auxiliary loss. Concretely, we bypass the fusion projection and set  $\mathbf{h}_t^{(K)} \triangleq \mathbf{m}_t^{(K)}$ , forcing the trunk to provide sufficient information for downstream decoding even without explicit dependence on the previous-level UA-SID. In DeepSeek-MTP, both inputs to the fusion layer contain only information from the previous token, and the earlier decoder layers are used solely to support decoding at the first step. (3)  $K$  is a tunable hyper-parameter that provides a flexible accuracy-efficiency trade-off without changing the model size. When  $K = 1$ , LazyAR reduces to naive autoregression. This design differs substantially from DeepSeek-MTP, where adding decoder layers for subsequent steps results in a substantial growth in model parameters. In our experiments,  $K = \frac{2}{3}L$  preserves recommendation quality while doubling inference throughput. (4) Furthermore, because the first  $K$  layers are computed only once and shared across beams, their impact on beam-search inference cost is significantly reduced. It suggests that these  $K$  layers can potentially be extended to perform more inference steps and incorporate latent reasoning, opening up additional capacity for improving model effectiveness. We leave this exploration to future work.

**Discussion.** This design is recommendation-specific and is less suitable for standard LLM decoding. In typical LLM serving, beam search is often not used (or uses a small beam), and the difficulty of predicting later tokens does not necessarily decrease, so deferring autoregressive dependency to later layers may yield limited speedup and offers no guarantee for long, variable-length generations.

### 3.3 Value-Aware Supervised Learning

We train GR4AD using a Value-Aware Supervised Learning (VSL) objective tailored to advertising recommendation. Similar to language models, the core supervision signal is defined on discrete tokens and optimized via cross-entropy loss. Given the UA-SID sequence  $\mathbf{y}$ , we first define a standard autoregressive token prediction

loss over SID tokens:

$$\mathcal{L}_{\text{SID}} = - \sum_{t=1}^T \log p(s_t | s_{<t}, \mathbf{X}). \quad (12)$$

*ECPM-Aware Token Prediction.* To better adapt the training objective to advertising scenarios, we further incorporate business value information by introducing an *eCPM token*, which can be used to re-rank the generated SIDs. Specifically, we discretize the continuous eCPM values of training samples into equiprobable buckets and append the resulting eCPM token as an additional prediction step after the UA-SID sequence. The eCPM prediction is optimized using a cross-entropy loss:

$$\mathcal{L}_{\text{eCPM}} = - \log p(v | \mathbf{y}, \mathbf{X}), \quad (13)$$

where  $v$  denotes the discretized eCPM token. The combined next-token prediction loss is then

$$\mathcal{L}_{\text{NTP}} = \mathcal{L}_{\text{SID}} + \lambda_e \mathcal{L}_{\text{eCPM}}. \quad (14)$$

*Value-Aware Sample Weighting.* In advertising, training samples exhibit highly skewed value distributions. To reflect their heterogeneous importance, we apply a value-aware weighting scheme to all loss terms. Each sample is assigned a weight  $w = w_{\text{user}} \cdot w_{\text{behavior}}$ , where  $w_{\text{user}}$  captures the long-term advertising value of the user, and  $w_{\text{behavior}}$  reflects the depth of user interaction (e.g., purchase actions receive higher weights than clicks). Samples from users with higher advertising value and deeper engagement thus contribute more strongly during training.

*Auxiliary MTP Loss.* As discussed in Section 3.2, LazyAR computes the first  $K$  decoder layers without conditioning on the previous token. To encourage these parallel layers to learn richer and more predictive representations, we introduce an auxiliary multi-token prediction (MTP) loss by setting  $\mathbf{h}_t^{(K)} \triangleq \mathbf{m}_t^{(K)}$ . Concretely, we require the trunk states to directly predict target tokens without relying on late-injected autoregressive signals. This auxiliary loss is applied during training only and is denoted as  $\mathcal{L}_{\text{MTP}}$ .

The final VSL objective is defined as

$$\mathcal{L}_{\text{VSL}} = \mathbb{E}_{\mathcal{D}} [w (\mathcal{L}_{\text{NTP}} + \lambda_{\text{mtp}} \mathcal{L}_{\text{MTP}})], \quad (15)$$

where  $\lambda_{\text{mtp}}$  controls the strength of the auxiliary MTP loss.

### 3.4 Ranking-Guided Reinforcement Learning

VSL enables GR4AD to learn a distribution over UA-SIDs that reflects historical user interests and advertising signals, but it mainly fits the logged data distribution and does not directly optimize downstream objectives. In particular, it does not explicitly favor UA-SIDs with higher advertising value or support exploration beyond observed behaviors. We therefore introduce a ranking-guided RL stage on top of VSL to encourage value-aware, list-level optimization while enabling controlled exploration, remaining grounded in the learned distribution.

**3.4.1 RSPO.** Unlike LLMs, recommendation systems aim to generate a ranked list rather than a single output, making per-item rewards insufficient to fully capture list-level optimization. Moreover, training samples in our setting are not limited to log feedback generated by the generative model itself, but also include samples collected from other production pipelines.

To address these challenges, we propose a list-wise RL method tailored to advertising recommendation, termed **RSPO** (Ranking-Guided Softmax Preference Optimization). Let  $\mathcal{Y} = \{y_1, y_2, \dots, y_n\}$  denote the candidate list, and  $v_i$  denotes the associated reward (eCPM) of  $y_i$ . Inspired by Lambda framework [29] and SDPO [4], instead of constructing chosen-rejected pairs using heuristic rules, RSPO directly aligns the RL objective with the ranking NDCG:

$$\mathcal{L}_{\text{RSPO}} = -\mathbb{E}_{(\mathbf{X}, \mathbf{y}_i, \mathcal{E}_i) \sim \mathcal{D}} \left[ \log_2 \sigma \left( -\log \sum_{y_j \in \mathcal{E}_i} \mathcal{M}_{ij} \exp \left( \beta \log \frac{p_{\theta}(\mathbf{y}_j | \mathbf{X})}{p_{\text{ref}}(\mathbf{y}_j | \mathbf{X})^{C_{ij}}} - \beta \log \frac{p_{\theta}(\mathbf{y}_i | \mathbf{X})}{p_{\text{ref}}(\mathbf{y}_i | \mathbf{X})^{C_{ij}}} \right) \right) \right]. \quad (16)$$

Here,  $\mathcal{E}_i = \{y_j | v_j < v_i\} \subset \mathcal{Y}$  denotes the set of candidates ranked below  $y_i$ . The coefficient  $\mathcal{M}_{ij} = \left| \frac{1}{D_{|i-j|}} - \frac{1}{D_{|i-j|+1}} \right| |G_i - G_j|$  follows the standard Lambda formulation, where  $G_i = \frac{2^{v_i} - 1}{Z}$ ,  $D_i = \log_2(1 + i)$ , and  $Z$  is the ideal DCG. We show that  $\mathcal{L}_{\text{RSPO}}$  is an upper bound of NDCGcost (proof in Appendix A.1) with respect to the ranking induced by  $\log \frac{p_{\theta}(\mathbf{y}_i | \mathbf{X})}{p_{\text{ref}}(\mathbf{y}_i | \mathbf{X})^{C_{ij}}}$ :

$$\text{NDCGcost} = \sum_{i=1}^n G_i - \sum_{i=1}^n \frac{G_i}{D_i} = \sum_{i=1}^n G_i - \text{NDCG}, \quad (17)$$

$\beta$  controls the preference strength and  $C_{ij}$  is a binary gate for reference availability and reliability. In production, training samples come from heterogeneous sources. Some lists are generated by GR4AD, for which we can record historical online predictions as a reference distribution  $p_{\text{ref}}$ ; however, many samples are collected from other pipelines and thus have no reliable  $p_{\text{ref}}$ . Moreover, even for GR4AD-generated samples,  $p_{\text{ref}}$  may become stale due to distribution drift and training-serving inconsistencies. When the current model predictions deviate too much from  $p_{\text{ref}}$ , enforcing the reference constraint can introduce noisy regularization and lead to unstable updates. We therefore enable  $p_{\text{ref}}$  only when it is available and deemed reliable; otherwise, we drop the reference:

$$C_{ij} = \begin{cases} 1 & \frac{1}{|\mathcal{E}_i \cup \{y_i\}|} \sum_{y_t \in \mathcal{E}_i \cup \{y_i\}} \left| \log \frac{p_{\theta}(\mathbf{y}_t | \mathbf{X})}{p_{\text{ref}}(\mathbf{y}_t | \mathbf{X})} \right| < \delta \\ 0 & \text{otherwise} \end{cases}, \quad (18)$$

where  $\delta$  is a hyperparameter threshold.

**3.4.2 Unified Learning of VSL and RSPO.** In LLM training, pre-training, supervised fine-tuning, and reinforcement learning are typically conducted in separate stages. In our production setting, however, GR4AD is updated continuously via online learning, making it essential to *jointly* integrate VSL and RSPO in a single training stream. Inspired by HPT [20], we treat VSL as learning a stable *base distribution* over user-interest items, while RSPO refines this distribution by biasing generation toward *higher-value* items without drifting away from user relevance.

To balance imitation and exploration dynamically, we introduce a sample-level *alignment score* that measures the mismatch between the model's current preference and the reward signal. For a candidate list of size  $n$ , let  $r_p^{(i)}$  and  $r_v^{(i)}$  be the ranks of candidate  $i$  according to the model likelihood  $p_{\theta}(\mathbf{y}_i | \mathbf{X})$  and its reward  $v_i$

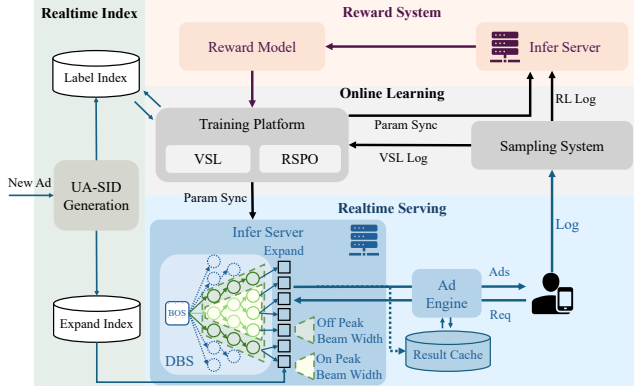


Figure 4: System overview: training and serving of GR4AD.

(eCPM), respectively. We define the normalized rank discrepancy:

$$A^{(i)} = \frac{|r_p^{(i)} - r_o^{(i)}|}{n-1}, \quad A^{(i)} \in [0, 1]. \quad (19)$$

When  $A^{(i)}$  is large, the model ranking deviates from the reward ranking, indicating insufficient imitation of the user-interest distribution; thus we increase the weight of VSL. When  $A^{(i)}$  is small, the model is already broadly aligned, and we place more emphasis on RSPO to improve value-aware listwise optimization.

We assign per-sample weights for the two objectives as:

$$w_{\text{VSL}}^{(i)} = w_0 \cdot \exp(A^{(i)} \cdot \log(1 + v_i)), \quad (20)$$

$$w_{\text{RL}}^{(i)} = w_0 \cdot Z_{\max}(1 - A^{(i)}), \quad (21)$$

where  $w_0$  is a base scaling factor and  $Z_{\max}$  caps the RL weight. The final unified online training objective is

$$\mathcal{L} = \mathbb{E}_{i \sim \mathcal{D}} \left[ w_{\text{VSL}}^{(i)} \mathcal{L}_{\text{VSL}}^{(i)} + w_{\text{RL}}^{(i)} \mathcal{L}_{\text{RSPO}}^{(i)} \right]. \quad (22)$$

## 4 Deployment

We have deployed GR4AD (0.16B) in Kuaishou’s advertising system, serving over 400 million users. The system employs a closed-loop architecture integrating reward estimation, online learning, and real-time indexing for continuous model evolution, while specific efficiency optimizations ensure high-throughput (500+ QPS per L20), low-latency (<100ms) inference under fluctuating traffic.

### 4.1 System Overview

As shown in Figure 4, GR4AD operates as a closed-loop system with four components: a *Realtime Serving* engine for request handling and ranking, a *Realtime Index* module for Item-SID mapping, an *Online Learning* module for continual VSL/RL updates, and a *Reward System* that provides value-based feedback to close the loop.

**Reward System.** The Reward System supplements online logs for RL training, since relying only on online exposed SL samples is (i) data-limited and insufficient to align optimization with revenue objectives (e.g., eCPM), and (ii) directly performing exploration in real-time serving can incur noticeable performance cost. We train a reward model on real exposure data and use it to score candidate sets generated by GR4AD. In the Reward System, latency constraints are relaxed, allowing GR4AD to generate more candidates with a

larger beam and to introduce controlled random exploration. The resulting value estimates (e.g., eCPM) are streamed as RL logs to support ranking-aware RSPO updates.

**Online Learning module.** The Online Learning Module processes real-time request and interaction streams to construct two distinct training signals: (i) it prioritizes positive engagement samples to generate VSL logs for modeling the user-interest distribution, and (ii) it samples a subset of high-value users—including cases where explicit interactions may be absent—and routes their contexts to the reward system for value estimation. The online learner performs continual mini-batch training for both VSL and RL, pushing updated parameters to the inference server in real time.

**Realtime Index module.** The Realtime Index Module replaces the legacy embedding-based retrieval pipeline to enable generative recommendation. In traditional DLRM systems, retrieval relies on embedding indexes that must be frequently rebuilt as models update, making index refresh typically minute-level. In contrast, our SID-based indexing is driven by a relatively stable content-to-UA-SID mapping and does not require frequent global rebuilds. When a new item arrives, we simply compute its UA-SID from content and update a bidirectional index (UA-SID  $\leftrightarrow$  Item ID) in seconds. This design substantially improves item freshness and cold-start coverage, while also ensuring training-serving consistency.

**Realtime Serving.** Realtime Serving handles user requests by invoking the inference server to return ranked ad lists, while simultaneously logging serving contexts and user feedback to close the loop. To maintain high-throughput, low-latency inference under traffic fluctuations, we apply Dynamic Beam Serving (DBS), result caching, and other concurrency optimizations.

### 4.2 Efficiency Optimization

To reconcile the high computational demands of generative recommendation with strict production latency constraints, we introduce a suite of optimizations.

**4.2.1 Dynamic Beam Serving (DBS).** We propose Dynamic Beam Serving (DBS) to improve the efficiency-effectiveness trade-off of multi-step decoding in real-time advertising.

**Dynamic Beam Width (DBW).** The number of returned candidates is determined by the beam width at the *final* step, while the overall decoding cost is dominated by the beam widths in *earlier* steps, since they control the number of hypotheses propagated to subsequent steps. Therefore, a fixed beam width across steps is often suboptimal under a constrained compute budget. Motivated by this, DBS adopts a *progressively increasing* beam schedule across steps, which reduces intermediate computation while preserving the final candidate quality.

**Traffic-Aware Adaptive Beam Search (TABS).** Moreover, request traffic in recommendation systems exhibits strong peak-off-peak cycles, and serving constraints are dictated by peak load. DBS therefore further adjusts the overall beam scale according to instantaneous traffic. Let  $Q_t$  denote the serving QPS at time  $t$  and  $B_{\text{base}}$  be the baseline beam setting. We adapt the active beam scale based on traffic intensity and available computational slack  $C_{\text{avail}}$  by  $B_t =$

**Table 1: Overall performance and ablation of GR4AD.**

| Model Settings                      | $\Delta$ Revenue<br>vs. Base | $\Delta$ QPS<br>vs. GR-Base |
|-------------------------------------|------------------------------|-----------------------------|
| <i>Baselines</i>                    |                              |                             |
| DLRM (Base)                         | –                            | –                           |
| OneRec-V2 [40] (GR-Base)            | +1.68%                       | –                           |
| <i>Tokenization Optimizations</i>   |                              |                             |
| + UA-SID                            | +1.92%                       | 0%                          |
| <i>Learning Optimizations</i>       |                              |                             |
| + VSL                               | +2.80%                       | -25%                        |
| + VSL + DPO [24]                    | +3.16%                       | -25%                        |
| + VSL + GRPO [26]                   | +3.21%                       | -25%                        |
| + VSL + RSPO                        | +3.86%                       | -25%                        |
| + Unified VSL & RSPO (UVR)          | +4.01%                       | -25%                        |
| <i>Serving Optimizations</i>        |                              |                             |
| + UVR + DBS                         | <b>+4.32%</b>                | +20%                        |
| + UVR + DBS + DeepSeek-MTP [5]      | +3.98%                       | <b>+117%</b>                |
| <b>GR4AD (+ UVR + DBS + LazyAR)</b> | <b>+4.28%</b>                | <b>+117%</b>                |

$B_{\text{base}} \cdot f(Q_t, C_{\text{avail}})$ . During off-peak periods (e.g.,  $Q_t < Q_{\text{threshold}}$ ), we increase  $B_t$  to leverage otherwise idle compute, enabling broader hypothesis exploration and improving ranking quality, while keeping peak-time latency and throughput within budget.

**4.2.2 Reco Result Cache.** A single user may issue multiple ad requests within a short time window, during which both user intent and the candidate ad pool typically remain stable. As a result, inference outcomes are expected to be largely consistent across these requests. By caching previously generated ad recommendations, subsequent requests within a bounded interval (e.g., one minute) can directly reuse cached results, significantly reducing inference resource consumption without degrading serving performance.

**4.2.3 Other Optimizations.** We propose Beam-Shared KV Caching to organize beams along the sequence dimension. This allows multiple beams to share a single encoder KV cache, eliminating redundant memory accesses and reducing the per-step KV read complexity from  $O(B \cdot L)$  to  $O(L)$ . For beam search, we introduce TopK Pre-Cut. It first selects  $k$  candidates in parallel from each beam of the previous step, then performs a global top- $k$  selection over the aggregated candidates. This reduces the search space, improves GPU parallelism, and maintains search correctness. Further, we reduce numerical precision from FP32 to FP8 [6], significantly lowering both computational redundancy and memory access overhead.

## 5 Experiments

### 5.1 Overall Performance

As shown in Table 1, GR4AD significantly outperforms these baselines in both inference efficiency and revenue, demonstrating its effectiveness. Specifically, the baselines include previous DLRM-based Kuaishou advertising platform, and OneRec-V2[40], a state-of-the-art generative recommendation model. Note that GR models are typically served on more powerful GPUs and with higher device utilization, whereas the DLRM-based production stack involves multiple models co-serving, making single-model QPS not directly comparable. Therefore, we report serving efficiency as relative QPS

improvements over the GR-Base setting for fair comparison. Moreover, we conduct several extensive ablation studies to assess the contribution of each component in GR4AD.

**Value-Aware Online Learning.** (1) First, we find that introducing VSL significantly boosts online revenue, demonstrating the effectiveness of the VSL module. Specifically, VSL, incorporating user and behavior weighting as well as eCPM token prediction, better aligns with advertising scenario requirements by enabling differentiated user modeling and directly targeting business objectives to maximize revenue. (2) However, while VSL learns a fixed data distribution in a point-wise manner, it lacks strong generalization ability and fails to further explore user preferences. To address this limitation, incorporating RSPO aligns generation probabilities with the relative ranking order in a list-wise manner, resulting in the most significant improvement among all optimization components. This demonstrates that RSPO can more comprehensively capture user interest and preferences compared to DPO[24] or GRPO[26], leading to enhanced business revenue. (3) Finally, rather than simply combining VSL and RSPO, we unify them through a sample-level training indicator, effectively leveraging the strengths of both. This approach not only stabilizes training in an online learning setting but also results in additional revenue.

**Dynamic Beam Serving.** As shown in Table 1, we implement the DBS mechanism to optimize inference efficiency and maximize revenue. First, DBW allows us to replace a fixed beam width across layers with a dynamic beam width (e.g., replacing 512-512-512 with 128-256-512), which reduces the overall computational load, significantly improves inference efficiency, and does not compromise revenue. Then, with the help of TABS, we increase the beam width by 60% during off-peak periods to improve revenue, while keeping the beam width unchanged during peak periods. The collaboration of both mechanisms leads to an optimized balance between revenue and efficiency.

**Lazy Autoregressive Decoder.** We observe that LazyAR results in a marginal performance decrease, yet it nearly doubles qps. This highlights the dual benefits of LazyAR: on the one hand, it improves inference efficiency by sharing the majority of decoder layers and enabling parallel computation to reduce the overall computational load, and on the other hand, it introduces an MTP-style auxiliary loss to enrich latent representations, ensuring that performance remains as unaffected as possible. In LazyAR, we configure the total number of decoder layers,  $L=9$ , with the first  $K=6$  layers shared across all beams. This favorable accuracy-efficiency trade-off allows GR4AD to effectively handle Kuaishou’s full advertising traffic.

**Business Indicators.** Beyond revenue, there is a significant 17.5% increase in ad delivery for small and medium-sized advertisers, marking a remarkable achievement. Additionally, due to more precise interest modeling, the generative ad system enhanced the user experience, leading to a 10.17% improvement in ad conversion rates. Among less active users, we still observed a 7.28% increase in conversion rates. We attribute these gains to improved generalization from content-based SIDs and a more real-time index that better supports cold-start items. Overall, GR4AD supports a healthier platform ecosystem, delivering a win-win-win outcome for the platform, advertisers, and users.



Figure 5: Scaling laws of model size and beam width.

## 5.2 Scaling Laws for GR4AD

To validate the scalability of our approach, we further investigate scaling effects from both model parameters and inference beam.

**5.2.1 Model Scaling.** We conduct a controlled set of online A/B tests across four model sizes, containing 0.03B, 0.08B, 0.16B, and 0.32B parameters, while keeping the inference beam width fixed at 512. We observe a clear and monotonic improvement in the revenue metric as model size increases. Larger models consistently achieve lower training loss, which is consistent with the observed online performance trend and suggests stronger representational power and generative modeling capacity. Specifically, the revenue lift steadily increases from +2.13% to +4.43%, demonstrating that scaling generative recommenders yields substantial and reliable gains in real-world advertising settings.

**5.2.2 Inference Scaling.** Beyond model size, we further investigate inference-time scaling by increasing the beam search width to enhance candidate exploration during generation. Online results show consistent gains with wider beams at a fixed model size of 0.16B: the revenue lift increases from +2.33% at beam width 128 to +4.21% at beam width 1024. These results indicate that stronger inference-time search can further unlock the potential of generative recommenders and translate into meaningful business impact. In production, we select the beam width based on latency and compute budgets to balance online gains against serving cost.

Overall, we observe a clear Scaling Law in both model size and beam width, which provides valuable insights for balancing resource consumption and performance gains.

## 5.3 Quality of UA-SID

**Embedding Optimizations.** A high-quality underlying embedding should effectively capture both an advertisement’s distinctiveness and the relationships among advertisements. Therefore, we constructed an offline test set to evaluate photo-to-photo recall: a retrieval is considered successful if the two photos belong to the same advertised product. As illustrated in Table 2, QARM[19] achieves the poorest performance because it summarizes only the video content without accounting for advertisement-specific information. Understanding these cues with Qwen3-VL-7B [2] yields a substantial recall improvement. Further, after applying instruction tuning

Table 2: Ablation of UA-SID.

| Tokenization Settings             | Offline Metrics |               |              |
|-----------------------------------|-----------------|---------------|--------------|
|                                   | R@1↑            | R@5↑          | R@10↑        |
| <i>Embedding Optimizations</i>    |                 |               |              |
| QARM [19]                         | 0.541           | 0.812         | 0.893        |
| Qwen3-VL-7B [2]                   | 0.769           | 0.948         | 0.977        |
| Qwen3-VL-7B + IT + CL (UAE)       | <b>0.896</b>    | <b>0.985</b>  | <b>0.994</b> |
| <i>Quantization Optimizations</i> |                 |               |              |
|                                   | Cpr↓            | Col↓          | Util↑        |
| RQ-Kmeans [19, 39]                |                 |               |              |
| (4096, 4096, 4096)                | 3.54            | 85.44%        | 0.10%        |
| RQ-Kmeans + MR                    |                 |               |              |
| (16384, 4096, 1024)               | 1.78            | 59.72%        | 0.20%        |
| RQ-Kmeans + MG + MR (UA-SID)      |                 |               |              |
| (16384, 4096, 1024*)              | <b>1.07</b>     | <b>18.26%</b> | <b>0.34%</b> |

(IT) and co-occurrence learning (CL) to Qwen3-VL-7B, we obtain the best recall results, underscoring the importance of end-to-end fine-tuning of advertisement embedding models. More clustering visualization results are shown in Appendix.

**Quantization Optimizations.** Under the same quantization space (i.e., the product of codebook sizes across all levels), we regard a SID scheme as better if it achieves a *lower compression ratio* and a *lower collision rate*, since this indicates higher code utilization and stronger item discriminability while still grouping semantically similar items. We report three metrics, compression ratio (Cpr), collision rate (Col) and codebook utilization (Util). The detailed computation is listed in Appendix. The ablation results are summarized in Table 2. We observe that, under a fixed quantization space, allocating larger codebooks to earlier levels and smaller ones to later levels (i.e., a multi-resolution design) improves codebook utilization and reduces collisions. Furthermore, applying a randomized hashing strategy at the final layer (denoted \* in table) further lowers the collision rate, and can also facilitate learning collaborative structure among items beyond pure semantic partitioning.

In summary, optimization of the embeddings and quantization yielded a +0.24% increase in revenue, as illustrated in Table 1.

## 6 Conclusions

In this paper, we presented GR4AD, a production-oriented generative recommender for large-scale, real-time advertising with online learning. GR4AD is co-designed across tokenization, architecture, learning and serving: UA-SID facilitates accurate advertisement representation; LazyAR improves the efficiency of short, multi-candidate generation by relaxing layer-wise dependencies while preserving effectiveness; VSL and the proposed RSPO align continual optimization with business value in non-stationary markets; and dynamic beam serving adapts decoding compute to traffic fluctuations and latency budgets. Large-scale online A/B tests show up to 4.2% ad revenue improvement over a strong DLRM baseline, with consistent gains from both model scaling and inference-time scaling, while achieving high-throughput real-time serving in a fully deployed system. These results highlight the promise of recommendation-native generative design for advertising and suggest future directions in more robust continual learning, cost-aware inference control, and broader constraint optimization in real-world deployments.

## Acknowledgments

We sincerely thank the following individuals (listed in alphabetical order) for their invaluable contributions: Caiyi Xu, Chen Yang, Chen Li, Fuxing Zhang, Haiping Xu, Hongtao Cheng, Jin Ouyang, Jinghui Jia, Jingshan Lv, Kang Sun, Lejian Ren, Qigen Hu, Xiang He, Xin Ku, Xinchun Luo, Yiyu Wang, Yongchuan Wang, Zhan Hu, Zhaojie Liu, Zhongteng Han

## References

- [1] Radford Alec, Narasimhan Karthik, Salimans Tim, and Sutskever Ilya. 2018. Improving language understanding by generative pre-training. (2018).
- [2] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. 2025. Qwen3-VL Technical Report. *CoRR* abs/2511.21631 (2025).
- [3] Ben Chen, Xian Guo, Siyuan Wang, Zihan Liang, Yue Lv, Yufei Ma, Xinlong Xiao, Bowen Xue, Xuxin Zhang, Ying Yang, Huangyu Dai, Xing Xu, Tong Zhao, Mingcan Peng, Xiaoyang Zheng, Chao Wang, Qihang Zhao, Zhixin Zhai, Yang Zhao, Bochao Liu, Jingshan Lv, Xiao Liang, Yuqing Ding, Jing Chen, Chenyi Lei, Wenwu Ou, Han Li, and Kun Gai. 2025. OneSearch: A Preliminary Exploration of the Unified End-to-End Generative Framework for E-commerce Search. *CoRR* abs/2509.03236 (2025).
- [4] Yuxin Chen, Junfei Tan, An Zhang, Zhengyi Yang, Leheng Sheng, Enzhi Zhang, Xiang Wang, and Tat-Seng Chua. 2024. On Softmax Direct Preference Optimization for Recommendation. In *NeurIPS*.
- [5] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. *CoRR* abs/2412.19437 (2024).
- [6] Maxim Fishman, Brian Chmiel, Ron Banner, and Daniel Soudry. 2025. Scaling FP8 training to trillion-token LLMs. In *ICLR*. OpenReview.net.
- [7] Kairui Fu, Tao Zhang, Shuwen Xiao, Ziyang Wang, Xinming Zhang, Chenchi Zhang, Yuliang Yan, Junjun Zheng, Yu Li, Zhihong Chen, Jian Wu, Xiangheng Kong, Shengyu Zhang, Kun Kuang, Yu-Gang Jiang, and Bo Zheng. 2025. FORGE: Forming Semantic Identifiers for Generative Retrieval in Industrial Datasets. *CoRR* abs/2509.20904 (2025).
- [8] Yuqian Fu, Tinghong Chen, Jiajun Chai, Xihuai Wang, Songjun Tu, Guojun Yin, Wei Lin, Qichao Zhang, Yuanheng Zhu, and Dongbin Zhao. 2025. SRFT: A Single-Stage Method with Supervised and Reinforcement Fine-Tuning for Reasoning. *CoRR* abs/2506.19767 (2025).
- [9] Chang Gao, Chujie Zheng, Xionghui Chen, Kai Dang, Shixuan Liu, Bowen Yu, An Yang, Shuai Bai, Jingren Zhou, and Junyang Lin. 2025. Soft Adaptive Policy Optimization. *CoRR* abs/2511.20347 (2025).
- [10] Gelfand and Alan E. 2000. Gibbs sampling. *Journal of the American statistical Association* 95, 452 (2000), 1300–1304.
- [11] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron C. Courville, and Yoshua Bengio. 2014. Generative Adversarial Nets. In *NIPS*. 2672–2680.
- [12] Ruidong Han, Bin Yin, Shangyu Chen, He Jiang, Fei Jiang, Xiang Li, Chi Ma, Mincong Huang, Xiaoguang Li, Chunzhen Jing, Yueming Han, MengLei Zhou, Lei Yu, Chuan Liu, and Wei Lin. 2025. MTGR: Industrial-Scale Generative Recommendation Framework in Meituan. In *CIKM*. ACM, 5731–5738.
- [13] Ruining He, Lukasz Heldt, Lichan Hong, Raghunandan H. Keshavan, Shifan Mao, Nikhil Mehta, Zhengyang Su, Alicia Tsai, Yueqi Wang, Shao-Chuan Wang, Xinyang Yi, Lexi Baugher, Baykal Kakici, Ed H. Chi, Cristos Goodrow, Ningren Han, He Ma, Rómer Rosales, Abby Van Soest, Devansh Tandon, Su-Lin Wu, Weilong Yang, and Yilin Zheng. 2025. PLUM: Adapting Pre-trained Language Models for Industrial-scale Generative Recommendations. *CoRR* abs/2510.07784 (2025).
- [14] Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising Diffusion Probabilistic Models. In *NeurIPS*.
- [15] Yupeng Hou, Jiacheng Li, Ashley Shin, Jinsung Jeon, Abhishek Santhanam, Wei Shao, Kaveh Hassani, Ning Yao, and Julian J. McAuley. 2025. Generating Long Semantic IDs in Parallel for Recommendation. In *KDD* (2). ACM, 956–966.
- [16] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvish Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. 2025. Generative Recommendation with Semantic IDs: A Practitioner's Handbook. In *CIKM*. ACM, 6420–6425.
- [17] Diederik P. Kingma and Max Welling. 2019. An Introduction to Variational Autoencoders. *Found. Trends Mach. Learn.* 12, 4 (2019), 307–392.
- [18] Shih-Yang Liu, Ta-Chi Yen, Cheng-Yu Hsieh, Yueh-Ning Chen, Chin-Hsuan Lin, Yu-Wei Chao, Tsu-Jui Fu, Shou-De Lin, Wan-Ting Hsu, Hsiang-Sheng Chiu, Pei-Fu Guo, Chen-Hsiang Yu, and Wen-Hsuan Li. 2026. GDPO: Group reward-Decoupled Normalization Policy Optimization for Multi-reward RL Optimization. *arXiv preprint arXiv:2601.05242* (2026).
- [19] Xinchun Luo, Jiangxia Cao, Tianyu Sun, Jinkai Yu, Rui Huang, Wei Yuan, Hezheng Lin, Yichen Zheng, Shiyao Wang, Qigen Hu, Changqing Qiu, Jiaqi Zhang, Xu Zhang, Zhiheng Yan, Jingming Zhang, Simin Zhang, Mingxing Wen, Zhaojie Liu, and Guorui Zhou. 2025. QARM: Quantitative Alignment Multi-Modal Recommendation at Kuaishou. In *CIKM*. ACM, 5915–5922.
- [20] Xingtai Lv, Yuxin Zuo, Youbang Sun, Hongyi Liu, Yuntian Wei, Zhekai Chen, Lixuan He, Xuekai Zhu, Kaiyan Zhang, Bingning Wang, Ning Ding, and Bowen Zhou. 2025. Towards a Unified View of Large Language Model Post-Training. *CoRR* abs/2509.04419 (2025).
- [21] Yu Meng, Mengzhou Xia, and Danqi Chen. 2024. SimPO: Simple Preference Optimization with a Reference-Free Reward. In *NeurIPS*.
- [22] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *NeurIPS*.
- [23] Haohao Qu, Wenqi Fan, Zihuai Zhao, and Qing Li. 2025. TokenRec: Learning to Tokenize ID for LLM-Based Generative Recommendations. *IEEE Trans. Knowl. Data Eng.* 37, 10 (2025), 6216–6231.
- [24] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *NeurIPS*.
- [25] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Mahesh Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. In *NeurIPS*.
- [26] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *CoRR* abs/2402.03300 (2024).
- [27] Zihua Si, Zhongxiang Sun, Jiale Chen, Guozhang Chen, Xiaoxue Zang, Kai Zheng, Yang Song, Xiao Zhang, Jun Xu, and Kun Gai. 2024. Generative Retrieval with Semantic Tree-Structured Identifiers and Contrastive Learning. In *SIGIR-AP*. ACM, 154–163.
- [28] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable Item Tokenization for Generative Recommendation. In *CIKM*. ACM, 2400–2409.
- [29] Xuanhui Wang, Cheng Li, Nadav Golbandi, Michael Bendersky, and Marc Najork. 2018. The LambdaLoss Framework for Ranking Metric Optimization. In *CIKM*. ACM, 1313–1322.
- [30] Yidan Wang, Zhaochun Ren, Weiwei Sun, Jiyuan Yang, Zhixiang Liang, Xin Chen, Ruobing Xie, Su Yan, Xu Zhang, Pengjie Ren, Zhumin Chen, and Xin Xin. 2024. Content-Based Collaborative Generation for Recommender Systems. In *CIKM*. ACM, 2420–2430.
- [31] Yejing Wang, Shengyu Zhou, Jinyu Lu, Ziwei Liu, Langming Liu, Maolin Wang, Wenlin Zhang, Feng Li, Wenbo Su, Pengjie Wang, Jian Xu, and Xiangyu Zhao. 2025. NEZHA: A Zero-sacrifice and Hyperspeed Decoding Architecture for Generative Recommendations. *CoRR* abs/2511.18793 (2025).
- [32] Yi Xu, Moyu Zhang, Chenxuan Li, Zhihao Liao, Haibo Xing, Hao Deng, Jinxin Hu, Yu Zhang, Xiaoyi Zeng, and Jing Zhang. 2025. MMQ: Multimodal Mixture-of-Quantization Tokenization for Semantic ID Generation and User Behavioral Adaptation. *CoRR* abs/2508.15281 (2025).
- [33] Xiaoyong Yang, Yadong Zhu, Yi Zhang, Xiaobo Wang, and Quan Yuan. 2020. Large Scale Product Graph Construction for Recommendation in E-commerce. *CoRR* abs/2010.05525 (2020).
- [34] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *ICML*. OpenReview.net.
- [35] Gaowei Zhang, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, and Ji-Rong Wen. 2024. Scaling Law of Large Sequential Recommendation Models. In *RecSys*. ACM, 444–453.
- [36] Jun Zhang, Yi Li, Yue Liu, Changping Wang, Yuan Wang, Yuling Xiong, Xun Liu, Haiyang Wu, Qian Li, Enming Zhang, Jiawei Sun, Xin Xu, Zishuai Zhang, Ruoran Liu, Suyuan Huang, Zhaoxin Zhang, Zhengkai Guo, Shuojin Yang, Meng-Hao Guo, Huan Yu, Jie Jiang, and Shi-Min Hu. 2025. GPR: Towards a Generative Pre-trained One-Model Paradigm for Large-Scale Advertising Recommendation. *CoRR* abs/2511.10138 (2025).
- [37] Wenhao Zhang, Yuexiang Xie, Yuchang Sun, Yanxi Chen, Guoyin Wang, Yaliang Li, Bolin Ding, and Jingren Zhou. 2025. On-Policy RL Meets Off-Policy Experts: Harmonizing Supervised Fine-Tuning and Reinforcement Learning via Dynamic Weighting. *CoRR* abs/2508.11408 (2025).

- [38] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting Large Language Models by Integrating Collaborative Semantics for Recommendation. In *ICDE*. IEEE, 1435–1448.
- [39] Guorui Zhou, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Qiang Luo, Qianqian Wang, Qigen Hu, Rui Huang, Shiyao Wang, Weifeng Ding, Wuchao Li, Xinchun Luo, Xingmei Wang, Zexuan Cheng, Zixing Zhang, Bin Zhang, Boxuan Wang, Chaoyi Ma, Chengru Song, Chenhui Wang, Di Wang, Dongxue Meng, Fan Yang, Fangyu Zhang, Feng Jiang, Fuxing Zhang, Gang Wang, Guowang Zhang, Han Li, Hengrui Hu, Hezheng Lin, Hongtao Cheng, Hongyang Cao, Huanjie Wang, Jiaming Huang, Jiapeng Chen, Jiaqiang Liu, Jinghui Jia, Kun Gai, Lantao Hu, Liang Zeng, Liao Yu, Qiang Wang, Qidong Zhou, Shengzhe Wang, Shihui He, Shuang Yang, Siyang Mao, Sui Huang, Tiantian He, Tingting Gao, Wei Yuan, Xiao Liang, Xiaoxiao Xu, Xugang Liu, Yan Wang, Yang Zhou, Yi Wang, Yiwu Liu, Yue Song, Yufei Zhang, Yunfeng Zhao, Zhixin Ling, and Ziming Li. 2025. OneRec-V2 Technical Report. *CoRR* abs/2508.20900 (2025).
- [40] Guorui Zhou, Hengrui Hu, Hongtao Cheng, Huanjie Wang, Jiaxin Deng, Jinghao Zhang, Kuo Cai, Lejian Ren, Lu Ren, Liao Yu, Pengfei Zheng, Qiang Luo, Qianqian Wang, Qigen Hu, Rui Huang, Ruiming Tang, Shiyao Wang, Shujie Yang, Tao Wu, Wuchao Li, Xinchun Luo, Xingmei Wang, Yi Su, Yunfan Wu, Zexuan Cheng, Zhanyu Liu, Zixing Zhang, Bin Zhang, Boxuan Wang, Chaoyi Ma, Chengru Song, Chenhui Wang, Chenglong Chu, Di Wang, Dongxue Meng, Dunju Zang, Fan Yang, Fangyu Zhang, Feng Jiang, Fuxing Zhang, Gang Wang, Guowang Zhang, Han Li, Honghui Bao, Hongyang Cao, Jiaming Huang, Jiapeng Chen, Jiaqiang Liu, Jinghui Jia, Kun Gai, Lantao Hu, Liang Zeng, Qiang Wang, Qidong Zhou, Rongzhou Zhang, Shengzhe Wang, Shihui He, Shuang Yang, Siyang Mao, Sui Huang, Tiantian He, Tingting Gao, Wei Yuan, Xiao Liang, Xiaoxiao Xu, Xugang Liu, Yan Wang, Yang Zhou, Yi Wang, Yiwu Liu, Yue Song, Yufei Zhang, Yunfeng Zhao, Zhixin Ling, and Ziming Li. 2025. OneRec-V2 Technical Report. *CoRR* abs/2506.13695 (2025).
- [41] Guorui Zhou, Shiyao Wang, Shucheng Li, Jiaxing Song, Yujie Lu, Weijieying Ren, Hao Liu, Bo Chen, Mingzhou Zhou, Yu Wang, Yiyang Zhang, Tianshu Wu, Jinze Bai, Xiang Li, Xiangyu Zhao, Xiuqiang He, Zhenhua Dong, Jieming Zhu, Ruiming Tang, Rui Zhang, Xi Xiao, Jun Xu, Zhengyan Zhang, Xuemin Zhao, Shuo Li, Zhiyuan Zhang, Ji-Rong Wen, Weinan Zhang, Tingting Zhang, Jun Wang, Xinxuan Chen, Xin Zhao, Lin Liu, Yifan Liu, Ruihua Song, Jian-Yun Nie, Hongning Wang, Dawei Yin, Hengshu Zhu, Hui Xiong, Depeng Jin, Yong Li, Wenwu Ou, Jian Pei, Bin Cui, and Ping Li. 2025. OpenOneRec Technical Report. *arXiv preprint arXiv:2512.24762* (2025).
- [42] Jieming Zhu, Mengqun Jin, Qijiong Liu, Zexuan Qiu, Zhenhua Dong, and Xiu Li. 2024. CoST: Contrastive Quantization based Semantic Tokenization for Generative Recommendation. In *RecSys*. ACM, 969–974.

## A Appendix

### A.1 Connection Between RSPO and NDCGcost

The definition of NDCG provided in the paper is restated as follows:

$$\text{NDCG} = \frac{1}{Z} \sum_{i=1}^n \frac{2^{v_i} - 1}{\log_2(1+i)} = \sum_{i=1}^n \frac{G_i}{D_i} \quad (23)$$

where  $Z$  is the ideal DCG,  $v_i$  denotes the associated reward (eCPM) for sample  $i$ ,  $G_i = \frac{2^{v_i} - 1}{Z}$ ,  $D_i = \log_2(1+i)$ . For any sample pair  $(y_i, y_j)$  with  $eCPM_i > eCPM_j$ . In RSPO, let the ranking score based on NDCG be denoted as:

$$\left\{ \frac{\log p_{\theta}(y_i | X)}{\log p_{ref}(y_i | X)^{C_{ij}}}, \frac{\log p_{\theta}(y_j | X)}{\log p_{ref}(y_j | X)^{C_{ij}}} \right\} = \{g_i, g_j\} \quad (24)$$

Following the LambdaLoss[29] framework, NDCGcost is defined as follows:

$$\begin{aligned} \text{NDCGcost} &= \sum_{i=1}^n G_i - \sum_{i=1}^n \frac{G_i}{D_i} \\ &= \sum_{i=1}^n G_i \sum_{j=1}^n \left| \frac{1}{D_{|i-j|}} - \frac{1}{D_{|i-j|+1}} \right| \mathbb{I}_{g_i < g_j} \\ &= \sum_{i=1}^n \sum_{y_j \in \mathcal{E}_i} \left| \frac{1}{D_{|i-j|}} - \frac{1}{D_{|i-j|+1}} \right| |G_i - G_j| \mathbb{I}_{g_i < g_j} + \text{Const} \\ &= \sum_{i=1}^n \sum_{y_j \in \mathcal{E}_i} \mathcal{M}_{ij} \mathbb{I}_{g_i < g_j} + \text{Const} \end{aligned} \quad (25)$$

Where  $\mathcal{M}_{ij} = \left| \frac{1}{D_{|i-j|}} - \frac{1}{D_{|i-j|+1}} \right| |G_i - G_j| \cdot \mathbb{I}_{g_i < g_j}$  is the zero-one indicator.  $\text{Const}$  is a constant, which is only related to the current ranking.  $\mathcal{E}_i$  denotes the set of negative samples corresponding to sample  $i$ , where  $v_j < v_i$ .

**THEOREM A.1.** Ignoring constant terms,  $\mathcal{L}_{\text{RSPO}}$  optimizes an upper bound of NDCGcost.

$$\begin{aligned} \text{NDCGcost} &= \sum_{i=1}^n \sum_{y_j \in \mathcal{E}_i} \mathbb{I}_{g_i < g_j} \mathcal{M}_{ij} \leq \sum_{i=1}^n \sum_{y_j \in \mathcal{E}_i} -\log_2 \sigma(g_i - g_j) \mathcal{M}_{ij} \\ &\leq \sum_{i=1}^n -\log_2 \sigma \left( -\log \sum_{y_j \in \mathcal{E}_i} \mathcal{M}_{ij} \exp(g_j - g_i) \right) = \mathcal{L}_{\text{RSPO}} \end{aligned} \quad (26)$$

**PROOF.** For the proof of the first inequality, we directly follow the inequality proposed in Lambdaloss, that is  $\mathbb{I}_{g_i < g_j} \leq -\log_2 \sigma(g_i - g_j)$ . For the proof of the second inequality, we prove the inequality for each sample  $i$  individually. Let  $\ell_i^{(1)}$  and  $\ell_i^{(2)}$  denote the inner terms for sample  $i$ , respectively. Using the identity  $-\log_2 \sigma(x) = \log_2(1 + e^{-x})$ , we rewrite  $\ell_i^{(1)}$  and  $\ell_i^{(2)}$ :

$$\ell_i^{(1)} = \sum_{y_j \in \mathcal{E}_i} \mathcal{M}_{ij} \log_2(1 + e^{-(g_i - g_j)}) = \sum_{y_j \in \mathcal{E}_i} \mathcal{M}_{ij} \log_2(1 + e^{g_j - g_i}) \quad (27)$$

$$\ell_i^{(2)} = -\log_2 \sigma \left( -\log \sum_{y_j \in \mathcal{E}_i} \mathcal{M}_{ij} e^{g_j - g_i} \right) = \log_2 \left( 1 + \sum_{y_j \in \mathcal{E}_i} \mathcal{M}_{ij} e^{g_j - g_i} \right) \quad (28)$$

To prove A.1, it is sufficient to demonstrate that  $\ell_i^{(1)} \leq \ell_i^{(2)}$ . According to the definition of  $\mathcal{M}_{ij}$ , we have:

$$\begin{aligned} \sum_{y_j \in \mathcal{E}_j} \mathcal{M}_{ij} &\leq \sum_{y_j \in \mathcal{E}_j} \left| \frac{1}{D_{|i-j|}} - \frac{1}{D_{|i-j|+1}} \right| \\ &= \sum_{k \in [b_1, b_2]} \left| \frac{1}{D_k} - \frac{1}{D_{k+1}} \right| = \left| \frac{1}{D_{b_1}} - \frac{1}{D_{b_2}} \right| \\ &= \left| \frac{1}{\log_2(1+b_1)} - \frac{1}{\log_2(1+b_2)} \right| < 1 \end{aligned} \quad (29)$$

Let  $k$  be the distance between  $i$  and  $j$ , such that  $k \in [b_1, b_2]$  with  $b_1, b_2 \geq 1$ . Introduce a dummy variable to normalize the weights. Let us define a complementary weight  $\mathcal{M}_{i0}$  such that:

$$\mathcal{M}_{i0} = 1 - \sum_{y_j \in \mathcal{E}_i} \mathcal{M}_{ij}. \quad (30)$$

Let  $x_j = e^{g_j - g_i}$ . Define the function  $f(x) = \log_2(1+x)$  for  $x \geq 0$ . Its second derivative is  $f''(x) = -\frac{1}{\ln 2(1+x)^2} < 0$ , which implies that  $f(x)$  is strictly **concave**. We apply **Jensen's Inequality** (where  $\mathbb{E}[f(X)] \leq f(\mathbb{E}[X])$ ) to the set of values  $\{x_j\}_{y_j \in \mathcal{E}_i} \cup \{x_0\}$ , where we choose  $x_0 = 0$ . The inequality becomes:

$$\begin{aligned} \mathcal{M}_{i0} \log_2(1+x_0) + \sum_{y_j \in \mathcal{E}_i} \mathcal{M}_{ij} \log_2(1+x_j) &\leq \\ \log_2 \left( 1 + \left( \mathcal{M}_{i0} x_0 + \sum_{y_j \in \mathcal{E}_i} \mathcal{M}_{ij} x_j \right) \right) \end{aligned} \quad (31)$$

Which is exactly:

$$\sum_{y_j \in \mathcal{E}_i} \mathcal{M}_{ij} \log_2(1+x_j) \leq \log_2 \left( 1 + \sum_{y_j \in \mathcal{E}_i} \mathcal{M}_{ij} x_j \right) \quad (32)$$

This completes the proof.  $\square$

### A.2 Efficiency Optimizations

Table 3 presents the key practical optimization points and their corresponding improvements in QPS.

**Table 3: Practical Inference Efficiency Optimizations**

| Key Optimization (vs. GR4AD) | $\Delta$ QPS |
|------------------------------|--------------|
| Beam-shared KV Cache         | +212.5%      |
| TopK Pre-Cut                 | +184.8%      |
| Low-Precision Inference      | +50.3%       |
| Reco Result Cache            | +27.8%       |

**KV Cache.** The KV Cache is a widely recognized technique used in the inference process of LLMs. During next-token prediction, it retains the keys and values of self/cross-attention from previous steps, significantly reducing redundant computation and memory access. This optimization is lossless with respect to model outputs and delivers a 2.5× throughput improvement over a non-cached baseline. Building on KV caching, our proposed Beam-shared KV Caching further increases queries per second (QPS) by an additional 25%, yielding an overall inference speedup of more than 3× compared with the non-cached baseline.

**TopK Pre-Cut.** : During beam search inference, at step  $i$  we select  $b_i$  candidates from  $b_{i-1} \times |V_i|$  possibilities. Instead of a direct global selection, we first select  $b_i$  candidates from  $|V_i|$  in parallel for each of the  $b_{i-1}$  beams, then choose the final  $b_i$  from the resulting  $b_{i-1} \times b_i$  candidates. Since  $|V_i| \gg b_i$  in practice, this increases GPU parallelism for top- $k$  and reduces comparisons, yielding significant speedups without changing the results.

**Low-Precision Inference.** During inference, we adopt FP8 precision [6] to replace FP32 computation, introducing negligible loss in output quality. A/B testing shows a marginal revenue change of approximately -0.1%, while inference throughput improves by 50.3%.

**Reco Result Cache.** We implement the result caching mechanism and serve 27.8% of requests directly from the cache within the defined time window (one minute), thereby improving the overall performance of the inference service.

**Dynamic Beam Width.** Our investigation into the effect of beam width across layers reveals that its influence on prediction performance escalates progressively. The model exhibits the highest prediction reliability at the initial layers, with later layers requiring wider beams for refined decision-making (Table 4). This insight directly motivates the design of our Dynamic Beam Width (DBW) mechanism, which allocates computational resources more efficiently across the model depth.

**Table 4: Comparison of Dynamic Beam Width Strategies**

| DBW Strategy           | Beam Width    | $\Delta$ Revenue | $\Delta$ QPS |
|------------------------|---------------|------------------|--------------|
| GR-Base                | [512,512,512] | –                | –            |
| 1st-level Reduction    | [128,512,512] | –0.10%           | +27%         |
| 2nd-level Reduction    | [512,128,512] | –0.23%           | +27%         |
| 3rd-level Reduction    | [512,512,128] | –0.85%           | +5%          |
| Progressive Increasing | [128,256,512] | –0.15%           | <b>+45%</b>  |

### A.3 Unified Advertisement SID

#### A.3.1 SID Evaluation Offline Metrics.

$$\text{Cpr} = \frac{\# \text{ Item}}{\# \text{ SID}}, \quad (33)$$

$$\text{Col} = 1 - \frac{\# \text{ one-on-one SID}}{\# \text{ SID}}, \quad (34)$$

$$\text{Util} = \frac{\# \text{ Item}}{\# \text{ Codebook Space}}, \quad (35)$$

where ‘one-on-one SID’ represents the SID associated with only one item.

#### A.3.2 Intruction Tuning.

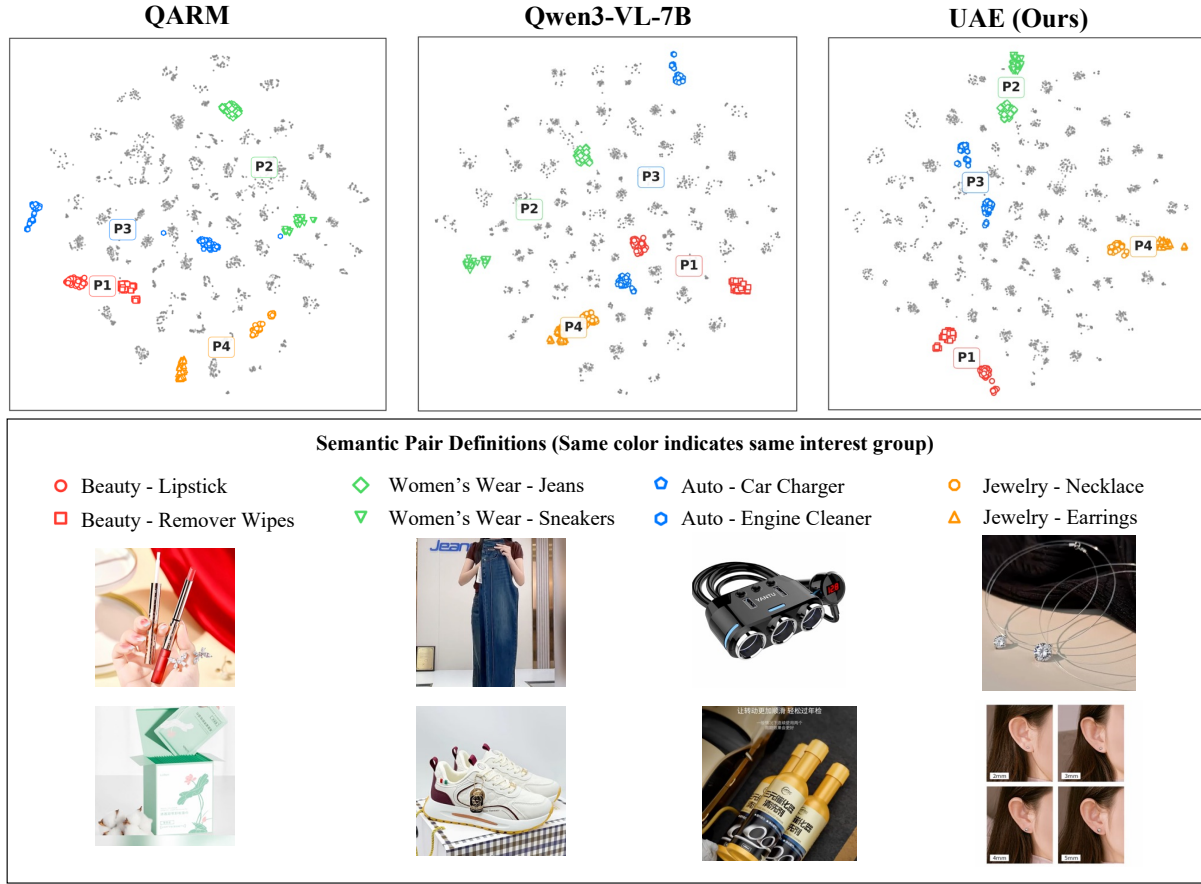
**Template 1.** Please perform deep semantic analysis on the input structured text (covering physical products, virtual services, and general entertainment content). Thoroughly fuse discrete fields such as product name, industry classification, detailed category, and brand to accurately determine the vertical domain. Ignore morphological differences and focus on extracting the core category identity and key value attributes (e.g., functional characteristics for physical goods or thematic intent for virtual services), constructing a general semantic representation that precisely defines their essence.

**Template 2.** Integrate livestream visual screenshots with multi-dimensional textual information (including streamer profile, regional attributes, livestream title, and user comments/interaction) for deep content understanding. Identify and extract the livestream’s core theme, the streamer’s persona/style, and the current interaction/engagement atmosphere, filtering out nonessential chatter and noise. Construct a semantic representation of the livestream scenario suitable for precise matching and retrieval.

**Template 3.** Combine the provided image sequence with product text fields to accurately identify and extract the product’s key identity features. Ignore irrelevant marketing phrasing and focus on the product’s essential attributes (industry, category, brand, and core specifications), constructing high-quality product feature representations for precise matching.

**Template 4.** Perform deep analysis of virtual products and content-service advertisements (including short dramas, games, apps, etc.). While accounting for differences in product form, focus uniformly on extracting two core feature types: the “core delivered value” and the “narrative marketing strategy.” Accurately identify whether the offering delivers emotional gratification, utilitarian functionality, or cognitive skill-building; and deeply decompose how conversion is driven through scenario construction (e.g., plot conflict, pain-point simulation) and psychological inducements (e.g., suspense hooks, vision framing). Filter out noise and construct a general semantic representation that precisely reflects its benefit attributes and marketing intent.

**Template 5.** Analyze advertising video materials intended to drive traffic to livestreams or directly promote products. Integrate visual dynamics and speech interactions to deeply decompose their traffic-conversion logic and persona atmosphere. Identify the video’s core driving elements: the streamer’s persona appeal, product demonstrations and endorsements, the rhetoric or plot devices used to create suspense/expectation (e.g., teasing major offers, emphasizing price advantages, creating urgency), and the livestream’s conveyed core value (premium assortment, cost-effectiveness, expertise, entertainment interaction). Construct semantic feature representations that accurately reflect the marketing and livestream conversion intentions.



**Figure 6: t-SNE visualization of different item embeddings. ‘P\*’ represents the clustering center of interest group.**

*Template 6.* Perform a multi-dimensional deep analysis of input physical-product advertisement videos by combining visual frames, scripted voiceover, and OCR-extracted marketing text. While identifying core product attributes (brand, function, appearance), emphasize analysis of content creativity and marketing techniques: determine the presentation format (e.g., narrative dramatization, stress testing, unboxing, factory-sourcing), extract key marketing hooks (e.g., time-limited discounts, pain-point reversals, buy-one-get-one offers), and characterize visual presentation style. Filter out irrelevant noise and construct a high-dimensional semantic

representation that encompasses both product features and content-marketing strategy.

**A.3.3 t-SNE Visualization.** Since QARM [19] lacks ad-specific product information, it can only infer relevance from visual signals, which prevents linking videos with large visual differences that refer to the same product. Qwen3-VL-7B [2] can extract some information but lacks fine-tuning on real-world application data and relational signals; after instruction tuning and co-occurrence learning, the resulting UAE effectively differentiates relationships across interest groups and, within a single interest group, more finely discriminates specific product categories, as illustrated in Figure 6.